

Architettura degli elaboratori e sistemi operativi

Diego Stefanini — prof. Marco Danelutto, a.a. 2026-27

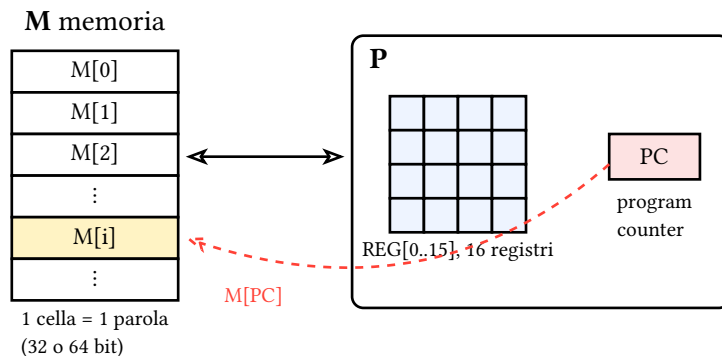
Indice

1	Fondamenti dei sistemi di elaborazione	3
1.1	Memoria, processore, istruzioni	3
1.2	Astrazione	4
1.3	Numeri binari	5
1.4	Basi, numeri reali e caratteri	9
2	Reti logiche	11
2.1	Porte logiche e tabelle di verità	11
2.2	Tempo di stabilizzazione	13
2.3	Componenti	14
2.4	Semplificare le formule	17

1 Fondamenti dei sistemi di elaborazione

1.1 Memoria, processore, istruzioni

Un calcolatore è fatto di due parti che si scambiano dati di continuo: la **memoria** M, che contiene programmi e dati, e il **processore** P, che esegue i programmi.



Il PC contiene l'indirizzo della prossima istruzione da prendere in memoria

- La **memoria** è un lungo vettore di celle numerate. Il numero di una cella è il suo **indirizzo**: $M[i]$ è la cella di indirizzo i . Ogni cella contiene una **parola** di 32 o 64 bit (un bit è una cifra 0 o 1, vedi i numeri binari più avanti).
- I **registri** sono poche celle (qui 16) **dentro** il processore: molto più veloci della memoria, ci stanno i dati su cui il processore sta lavorando in quel momento.
- Il **PC** (*program counter*) è un registro speciale: contiene l'indirizzo della prossima istruzione da eseguire. Anche il programma, infatti, sta in memoria.

Una volta avviato, il processore ripete per sempre questi quattro passi, il **ciclo fetch-decode-execute**:

```
while (true) {  
    preleva l'istruzione da M[PC]  
    decodifica      // che istruzione è?  
    esegue  
    scrive i risultati  
}
```

Dopo il prelievo il PC passa all'istruzione successiva, così al giro dopo il processore prende quella dopo.

Quali istruzioni esistono dipende dal processore (x86, ARM, RISC-V sono famiglie di processori diverse), ma i tipi sono sempre tre:

Tipo	Istruzioni	Esempio
operative	ADD SUB MUL DIV AND OR NOT SHIFT ROT (sx/dx)	ADD R0, R1, R2 → $REG[0] = REG[1] + REG[2]$ SHIFT sposta i bit a sinistra o a destra; ROT li ruota: il bit che esce da una parte rientra dall'altra
memoria	LOAD STORE	LOAD R5, addr → $REG[5] = M[addr]$ (addr = un indirizzo) STORE fa il contrario: registro → memoria
salto	salto	cambia il PC: la prossima istruzione non è quella dopo

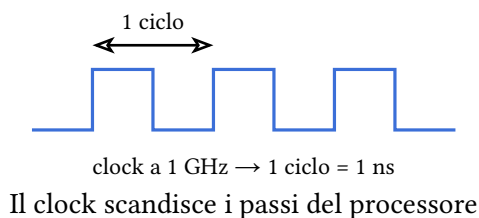
```

inizio:  ADD R0, R0, R0    ← PC
        ADD R1, R1, R1
        SUB R2, R0, R1
        salto inizio

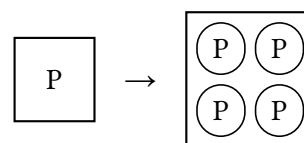
```

Il PC scende di un'istruzione alla volta; il **salto** lo rimette su `inizio`, quindi il programma ricomincia.

A dare il ritmo ai passi del ciclo è il **clock**, un segnale che passa da 0 a 1 e viceversa a intervalli regolari. Ogni passo del processore dura un certo numero di cicli di clock: più cicli al secondo (la **frequenza**, in Hz), più istruzioni al secondo.



- 1 GHz = 10^9 cicli al secondo, quindi un ciclo dura 10^{-9} s = 1 ns.
- Il processore è fatto di **transistor**, minuscoli interruttori. Nel Motorola 68000 (fine anni '70) misuravano 20-40 μm , oggi 2 nm ($1000 \text{ nm} = 1 \mu\text{m}$).
- Transistor più piccoli → nello spazio di un processore ne stanno più di uno. Ogni processore sul chip si chiama **core**, il chip è **multicore**:



Da sapere — legge di Moore (aggiunto, non spiegato in aula)

Il numero di transistor che si riescono a mettere su un chip raddoppia circa ogni due anni. È per questo che si passa dai μm ai nm e da un processore a più processori sullo stesso chip.

1.2 Astrazione

Un calcolatore si studia a **livelli di astrazione**: ognuno usa quello sotto senza doverne conoscere i dettagli.

Applicazioni
Sistema operativo — processi, memoria, file system
Architettura — set di istruzioni (ARMv7)
Microarchitettura — il processore
Logica — componenti: AND, OR, NOT, «memoria»
Circuiti digitali — segnali «1» e «0»: FA, MUX
Circuiti analogici
Dispositivi
Fisica — questi ultimi 3 non li facciamo

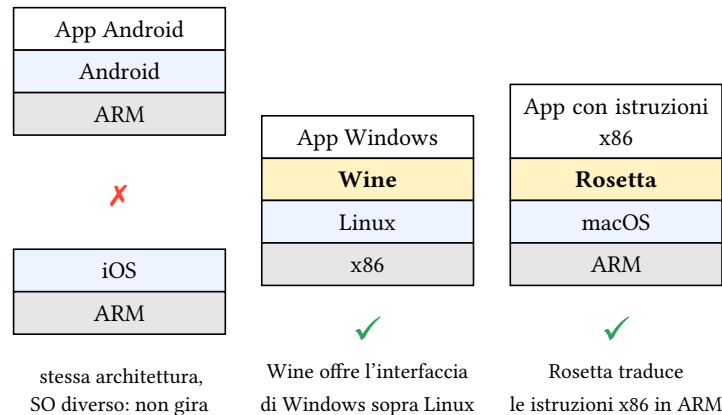
Il corso parte dal basso e risale. FA e MUX sono due circuiti che arrivano più avanti: il **full adder** somma una cifra, il **multiplexer** sceglie fra due segnali.

Applicazione — livello i+1
interfaccia: chiamate al sistema operativo
Sistema operativo — livello i
interfaccia: istruzioni in linguaggio macchina
Architettura — livello i-1

Fra due livelli c'è un'**interfaccia**: l'insieme delle funzionalità che il livello i-1 mette a disposizione del livello i.

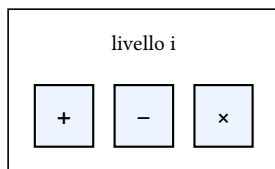
Il livello i usa solo l'interfaccia, non sa com'è fatto sotto.

Conseguenza: un programma gira solo dove trova l'interfaccia per cui è stato scritto. Un'app Android usa le chiamate di sistema di Android: su iOS non ci sono, anche se il processore (ARM) è lo stesso. Al contrario, basta che qualcuno offra l'interfaccia giusta, anche «finta», e il programma gira:

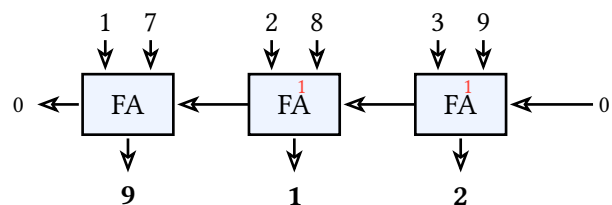


Per costruire sistemi così complessi si usano tre principi:

- **Gerarchia:** il sistema è diviso in livelli (sopra).
- **Modularità:** ogni livello è fatto di moduli con un compito preciso.
- **Regolarità:** lo stesso modulo si riusa tante volte.



Modularità: un livello fatto di moduli

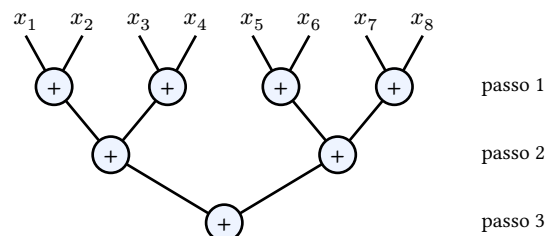


Regolarità: $123 + 789 = 912$ con tre FA uguali in cascata. Ogni FA somma una cifra e passa il riporto a sinistra

Stesso modulo **+**, due modi di sommare 8 numeri:

```
sum = 0;
for (i = 0; i < n; i++)
    sum = sum + x[i];
```

7 somme **una dopo l'altra**: ognuna aspetta il risultato della precedente, quindi 7 passi.



Ad albero: le somme dello stesso livello sono indipendenti, quindi si possono fare insieme (con 4 sommatore); le stesse 7 somme in 3 passi

1.3 Numeri binari

I circuiti digitali lavorano in **binario**: ogni segnale vale 0 o 1.

In un sistema posizionale **conta l'ordine delle cifre**: la stessa cifra vale di più quanto più è a sinistra. Ogni posizione ha un peso.

decimale	$10^3 = 1000$	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
binario	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

Esempio: $13_{10} \neq 31_{10}$ e $10_2 \neq 01_2$ (il numero piccolo in basso è la base).

Per passare **da binario a decimale** sommo i pesi delle posizioni dove c'è 1: $1100_2 = 8 + 4 = 12$.

Per passare **da decimale a binario** divido per 2 finché arrivo a 1, poi leggo i resti **dal basso verso l'alto**. Il resto della divisione per 2 dice se il numero è pari o dispari, cioè l'ultima cifra binaria; dividendo, «tolgo» quella cifra e passo alla successiva. Per questo le cifre escono da destra a sinistra.

n	: 2	resto
12	6	0
6	3	0
3	1	1
1		1 ↑

$$12 = 1100_2$$

n	: 2	resto
7	3	1
3	1	1
1		1 ↑

$$7 = 111_2$$

Somma e prodotto in binario si fanno come in decimale, ma $1 + 1 = 10_2$: scrivo 0 e riporto 1.

$$\begin{array}{r} 8421 \\ 1010 \\ + 0100 \\ \hline 1110 \end{array}$$

$10 + 4 = 14$ (pesi in grigio)

$$\begin{array}{r} 111 \\ 0111 \\ + 0001 \\ \hline 1000 \end{array}$$

$7 + 1 = 8$ (riporti in grigio)

Nel prodotto in colonna la cifra del moltiplicatore è 0 o 1, quindi ogni riga è tutta zeri oppure il numero stesso, spostata di un posto a sinistra (il $\cdot$ segna il posto lasciato vuoto).

$$\begin{array}{r} 12 \\ \times 34 \\ \hline 48 \\ 36 \cdot \\ \hline 408 \end{array} \quad \begin{array}{r} 1010 \\ \times 0100 \\ \hline 0000 \\ 0000 \cdot \\ 1010 \cdot\cdot \\ 0000 \cdot\cdot\cdot \\ \hline 0101000 \end{array}$$

in decimale

$$10 \times 4 = 40 = 32 + 8$$

Per i **numeri negativi** il modo più semplice è **modulo e segno**: il primo bit è il **segno** ($0 = +$, $1 = -$), gli altri sono il **valore assoluto**.

1	1	1	0	0
---	---	---	---	---

segno

valore assoluto = 12

−12 in modulo e segno

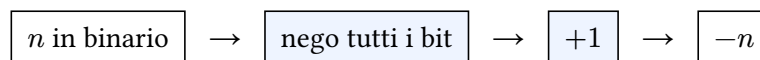
Per sommare A e B bisogna prima guardare i segni (S = segno, V = valore assoluto):

Caso	Risultato
$S_A = S_B$	segno S_A , valore $V_A + V_B$
$S_A \neq S_B$ e $V_A > V_B$	segno S_A , valore $V_A - V_B$
altrimenti	segno S_B , valore $V_B - V_A$

Esempi: $+12 + 12 = +24$, $-12 + (-12) = -24$, $-12 + 7 = -5$ (segni diversi: faccio $12 - 7$ e tengo il segno del più grande).

Servono confronti e sottrazioni: il circuito che somma diventa complicato. Il complemento a 2 risolve questo problema.

Si usa quindi il **complemento a 2**: i positivi si scrivono come sempre, e per passare da n a $-n$ si negano tutti i bit ($0 \leftrightarrow 1$) e si somma 1:



Vale anche al contrario: rifacendo «nego + 1» su $-n$ torno a n . Così, se un risultato comincia con 1 (è negativo), capisco quanto vale.

Il vantaggio: **la somma si fa come una somma normale**, senza guardare i segni.

Esempio: $-12 + 7$ su 5 bit.

$\begin{array}{r} 12 \quad 01100 \\ \text{nego} \quad 10011 \\ \hline +1 \quad \mathbf{10100} \\ \hline -12 \end{array}$	$\begin{array}{r} 10100 \\ + \quad 00111 \\ \hline \mathbf{11011} \\ \hline -12 + 7 \end{array}$	$\begin{array}{r} \text{somma} \quad 11011 \\ \text{nego} \quad 00100 \\ \hline +1 \quad \mathbf{00101} \\ \hline \end{array}$ inizia per 1, è negativo: vale -5 ✓
--	--	---

Esempio: $-5 + (-3)$ su 6 bit.

$\begin{array}{r} 5 \quad 000101 \\ \text{nego} \quad 111010 \\ \hline +1 \quad \mathbf{111011} \\ \hline -5 \end{array}$	$\begin{array}{r} 3 \quad 000011 \\ \text{nego} \quad 111100 \\ \hline +1 \quad \mathbf{111101} \\ \hline -3 \end{array}$	$\begin{array}{r} 11111 \\ 111011 \\ + \quad 111101 \\ \hline \mathbf{111000} \\ \hline \end{array}$ esce un riporto 1 oltre i 6 bit: si butta	$\begin{array}{r} \text{somma} \quad 111000 \\ \text{nego} \quad 000111 \\ \hline +1 \quad \mathbf{001000} \\ \hline \end{array}$ vale 8, quindi il risultato è -8 ✓
---	---	---	---

Nell'esempio $-5 + (-3)$ esce un riporto oltre i 6 bit e il risultato è giusto lo stesso: **il riporto in uscita non è un errore**. L'errore vero è l'overflow, qui sotto.

Quanti numeri ci stanno in n bit? Ogni bit raddoppia le combinazioni: $2 \cdot 2 \cdot 2 \dots = 2^n$. Con 8 bit sono $2^8 = 256$ combinazioni, e il bit in posizione i (contando da destra, da 0) pesa 2^i .

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

binario	valore
0000 0000	0
0000 0001	1
0000 0010	2
0000 0011	3
0000 0100	4
⋮	⋮
1111 1111	255

Le 256 combinazioni si possono usare in due modi:

con n bit	intervallo	8 bit
interi senza segno (solo +)	$0 \dots 2^n - 1$	$0 \dots 255$
relativi (+ e -) in complemento a 2	$-2^{n-1} \dots + 2^{n-1} - 1$	$-128 \dots + 127$

In complemento a 2 metà delle combinazioni va ai negativi e metà a zero e positivi: per questo c'è un negativo in più (-128) e i positivi si fermano a $+127$.

Un numero in complemento a 2 si legge così: **il bit più a sinistra pesa -2^{n-1}** , gli altri pesano $+2^i$ come sempre. Il segno è già dentro i pesi: per questo basta la somma normale.

-128	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$$1111\ 1101 = -128 + 64 + 32 + 16 + 8 + 4 + 1 = -3$$

valore	8 bit
3	0000 0011
2	0000 0010
1	0000 0001
0	0000 0000
-1	1111 1111
-2	1111 1110
-3	1111 1101
⋮	⋮
-128	1000 0000

- Tutti i negativi cominciano per 1, zero e positivi per 0.
- -1 è tutti 1: $0000\ 0001 \rightarrow$ nego $1111\ 1110 \rightarrow +1 \rightarrow 1111\ 1111$.
- -128 non ha l'opposto: facendo «nego + 1» torna se stesso, perché $+128$ non ci sta in 8 bit.

$$\begin{array}{r} -128\ 1000\ 0000 \\ \text{nego}\ 0111\ 1111 \\ \hline +1\ 1000\ 0000 \end{array}$$

In **modulo e segno** l'intervallo è $-(2^{n-1} - 1) \dots + (2^{n-1} - 1)$ e lo zero ha **due** rappresentazioni, $+0 = 0000\ 0000$ e $-0 = 1000\ 0000$. Anche un semplice `if (x == 0)` deve controllarle tutte e due: un altro motivo per cui si usa il complemento a 2.

Sommando due numeri di n bit il risultato può aver bisogno di $n + 1$ bit, ma il circuito ne ha solo n . Quando il risultato esce dall'intervallo rappresentabile si ha **overflow** (traboccamento).

Esempi su 3 bit, dove il complemento a 2 va da -4 a $+3$:

-4	-3	-2	-1	0	1	2	3
100	101	110	111	000	001	010	011

$$\begin{array}{r}
 1 \\
 010 \\
 + 010 \\
 \hline
 100
 \end{array}
 \qquad
 \begin{array}{r}
 101 \\
 + 110 \\
 \hline
 (1)011
 \end{array}$$

$2 + 2 = 4$, ma 100 vale -4 ✗ $-3 + (-2) = -5$, ma 011 vale $+3$ ✗

Regola: operandi con lo **stesso segno** e risultato con **segno diverso** $\Rightarrow$ **overflow**.

Con segni diversi l'overflow non può mai capitare: il risultato sta sempre fra i due operandi.

Il complemento a 2 permette di fare anche la **sottrazione con lo stesso sommatore**: $a - b = a + (-b)$.
 Il circuito è fra i componenti delle reti logiche.

1.4 Basi, numeri reali e caratteri

Un sistema posizionale in **base B** usa le cifre da 0 a $B - 1$, e la cifra in posizione i pesa B^i .

base	cifre	nota
$B = 10$	0, 1, ..., 9	
$B = 2$	0, 1	
$B = 8$	0, 1, ..., 7	una cifra = 3 bit
$B = 16$	0, ..., 9, A, B, C, D, E, F	una cifra = 4 bit (nibble); A = 10, ..., F = 15

In base 16 i pesi sono $16^2 = 256$, $16^1 = 16$, $16^0 = 1$. L'esadecimale si usa perché da binario si passa **senza conti**: $16 = 2^4$, quindi una cifra esadecimale corrisponde esattamente a 4 bit. Divido i bit in gruppi di 4 partendo da destra e scrivo ogni gruppo come una cifra. Il prefisso **0x** indica un numero esadecimale.

$$\begin{array}{cccc}
 \overbrace{0 \ 1 \ 1 \ 0}^6 & \overbrace{1 \ 1 \ 0 \ 1}^D & & 0110 \ 1101 = 0x6D = 64 + 32 + 8 + 4 + 1 = 109 \\
 \text{128} \ \text{64} \ \text{32} \ \text{16} & \text{8} \ \text{4} \ \text{2} \ \text{1} & &
 \end{array}$$

Esempio: -3 su 16 bit.

$$\begin{array}{r}
 3 \quad 0000 \ 0000 \ 0000 \ 0011 \\
 \text{nego} \quad 1111 \ 1111 \ 1111 \ 1100 \\
 +1 \quad \hline
 1111 \ 1111 \ 1111 \ 1101
 \end{array}
 \qquad
 \begin{array}{l}
 \text{ogni gruppo 1111 è una F, 1101 è D:} \\
 -3 = 0xFFFFD
 \end{array}$$

Per i **numeri con la virgola** la notazione posizionale continua a destra della virgola con esponenti negativi:

$$123,45 = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

Tenendo la virgola in una posizione fissa, per sommare $123,45 + 17,9$ bisogna prima allinearla (017,90). Si usa invece la **virgola mobile (floating point)**: cifre ed esponente separati, come nella notazione **mmmEn** = $mmm \times 10^n$.

$$\begin{array}{ll}
 12345E-2 & = 123,45 \quad (\text{la virgola si sposta di 2 a sinistra}) \\
 123E2 & = 12300
 \end{array}$$

Il numero si scrive come $\pm 0, \text{mantissa} \times 10^E$ e si salvano tre campi, in 32 o 64 bit:

S	E	mantissa
segno	esponente	valore 0,xxxx

$$\begin{array}{l}
 123,45 = +0,12345 \times 10^3 \\
 S = 0, \ E = 3, \ \text{mantissa} = 12345
 \end{array}$$

L'esponente può essere negativo: con 8 bit ha 256 valori, da -128 a $+127$. Così con pochi bit si scrivono numeri molto grandi e molto piccoli. Si chiama virgola **mobile** proprio perché cambiando l'esponente la virgola si sposta.

Anche i **caratteri** sono numeri. Il codice **ASCII** assegna un numero a ogni carattere, con 7 o 8 bit per carattere (cioè 128 o 256 caratteri); deve rappresentare:

- le 26 lettere, maiuscole e minuscole (e quelle accentate);
- le 10 cifre e la punteggiatura;
- i caratteri speciali: CR (ritorno a inizio riga), LF (riga nuova), TAB, DEL, ...

carattere	A	B	a	c	m
codice	65	66	97	99	109

Le lettere hanno codici in ordine alfabetico, quindi confrontare due stringhe (in C `char *s1, *s2`) vuol dire confrontare i codici carattere per carattere: «ciao» viene prima di «mondo» perché `c` = 99 < `m` = 109.

2 Reti logiche

2.1 Porte logiche e tabelle di verità

Una **rete combinatoria** è un circuito che calcola una funzione: l'uscita dipende solo dal valore degli ingressi in quel momento.

I circuiti digitali lavorano su $\{0, 1\}$ con tre operazioni di base, le **porte logiche**. Gli ingressi entrano da sinistra, l'uscita esce a destra.



AND			OR			NOT	
x	y	z	x	y	z	x	z
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

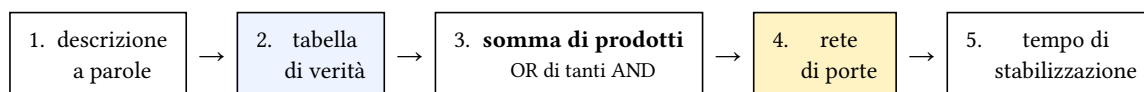
AND vale 1 solo se **tutti** gli ingressi valgono 1; OR vale 1 se **almeno uno** vale 1; NOT inverte. Nelle formule si usa una notazione più corta, come nell'algebra:

	AND	OR	NOT
si scrive	$x \cdot y$ oppure xy (prodotto)	$x + y$ (somma)	$\bar{x}$

Somiglia davvero ad aritmetica: su 0 e 1, AND è il prodotto. OR è la somma, tranne $1 + 1$ che fa 1. Alcune proprietà:

associativa	$(x \text{ AND } y) \text{ AND } z \equiv x \text{ AND } (y \text{ AND } z)$	
commutativa	$x \text{ AND } y \equiv y \text{ AND } x$	
elemento neutro	$x \text{ AND } 1 \equiv x$	$x \text{ OR } 0 \equiv x$
elemento assorbente	$x \text{ AND } 0 \equiv 0$	$x \text{ OR } 1 \equiv 1$

Per costruire una rete da zero si seguono cinque passi, il **procedimento standard**:



La tabella di verità è la funzione descritta caso per caso: per ogni combinazione di ingressi dice quanto vale l'uscita.

Esempio: il **MUX**. A parole: *scegli fra due ingressi x e y a seconda di un ingresso di controllo c .*

```
if (c == 0) z = x;
else      z = y;
```

Dalla descrizione si ricava la tabella di verità. Poi, per ogni riga dove $z = 1$, scrivo un prodotto (AND) che vale 1 **solo** in quella riga: la variabile così com'è se vale 1, negata ($\bar{x}$) se vale 0.

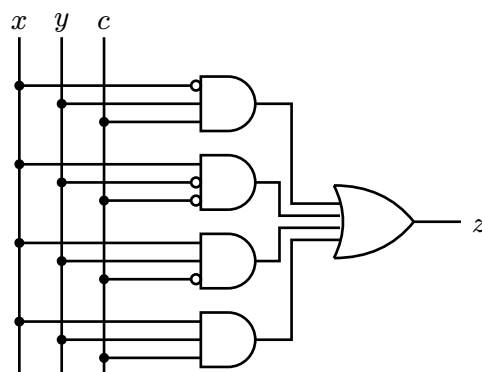
x	y	c	z	prodotto
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\bar{x} y c$
1	0	0	1	$x \bar{y} \bar{c}$
1	0	1	0	
1	1	0	1	$x y \bar{c}$
1	1	1	1	$x y c$

Le righe con $z = 1$ sono unite da un OR. Ogni prodotto vale 1 solo nella sua riga, quindi l'OR vale 1 esattamente nelle righe scelte.

$$z = \bar{x} y c + x \bar{y} \bar{c} + x y \bar{c} + x y c$$

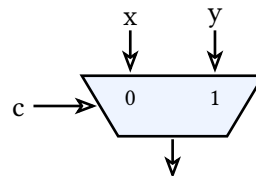
Per esempio $\bar{x} y c = \text{NOT}(x) \text{ AND } y \text{ AND } c$: vale 1 solo per $x = 0, y = 1, c = 1$.

Ogni prodotto diventa una porta AND a tre ingressi (il pallino sull'ingresso è un NOT), e le quattro uscite entrano in una OR:

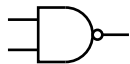


Il MUX come rete di porte

... che si disegna col simbolo



Le tre porte si possono ottenere tutte da una sola, la **NAND**: un AND con l'uscita negata (il pallino in fondo).

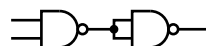


x	y	NAND
0	0	1
0	1	1
1	0	1
1	1	0

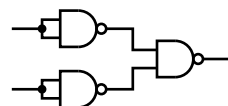
- **NOT**: $\text{NAND}(x, x)$, perché $x \text{ AND } x = x$ e poi viene negato.
- **AND**: una NAND seguita da un NOT, che toglie la negazione.
- **OR**: per la legge di **De Morgan** $x + y = \overline{\bar{x} \cdot \bar{y}}$, cioè nego i due ingressi e li mando in una NAND.



NOT: 1 NAND



AND: 2 NAND

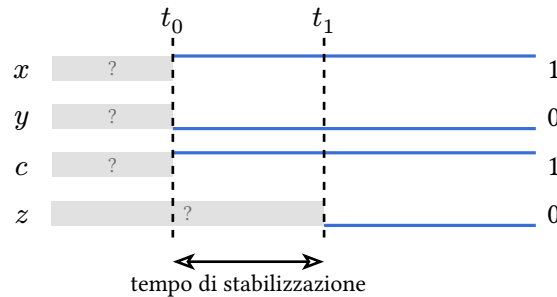


OR: 3 NAND

Con un solo tipo di porta si risparmia lavoro di progettazione, ma non spazio sul chip: per ogni AND o OR servono più NAND.

2.2 Tempo di stabilizzazione

Una porta non risponde subito. Da quando gli ingressi hanno il loro valore (istante t_0) a quando l'uscita è quella giusta (t_1) passa del tempo, e nel frattempo l'uscita può valere qualunque cosa. È il **tempo di stabilizzazione** (o ritardo) della rete.



Il MUX con $c = 1$ fa passare $y = 0$, ma z è affidabile solo da t_1 in poi

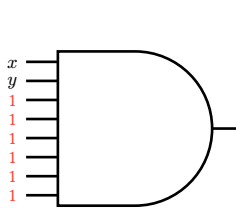
Il ritardo di una porta dipende da quanti ingressi ha: resta basso fino a una certa soglia, poi esplode. Nel corso si usa questa convenzione:



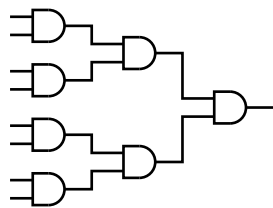
- AND e OR **fino a 8 ingressi**: ritardo Δt
- NOT: ritardo **0**

Il NOT non costa perché il segnale negato è già pronto dentro la porta che lo produce: basta prenderlo da un altro punto del circuito.

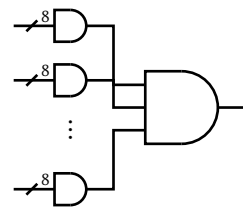
Con le porte da 8 si fanno anche quelle più piccole: gli ingressi che avanzano si collegano al valore che non cambia il risultato, 1 per l'AND e 0 per l'OR. Per più di 8 ingressi si collegano più porte **ad albero**, e ogni livello dell'albero costa Δt .



AND da 2 con una porta da 8:
sei ingressi fissi a 1



AND da 8 con porte da 2:
 $\log_2 8 = 3$ livelli

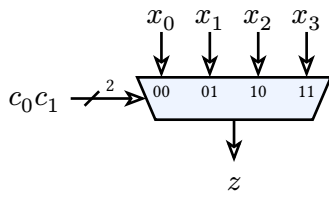


AND da 64 con porte da 8:
 $\log_8 64 = 2$ livelli

In generale, una porta da n ingressi fatta con porte da k ingressi richiede $\lceil \log_k n \rceil$ livelli ($\lceil \cdot \rceil$ = arrotondato per eccesso): la base è il numero di ingressi di una porta, l'argomento il numero di ingressi totali.

Il tempo di stabilizzazione di una rete si conta sui **livelli** di AND e OR che il segnale attraversa: le porte dello stesso livello lavorano insieme e costano un solo Δt , i livelli uno dopo l'altro si sommano. Il MUX ha un livello di AND e uno di OR: $2\Delta t$. Vale per ogni somma di prodotti, finché termini e variabili non sono più di 8.

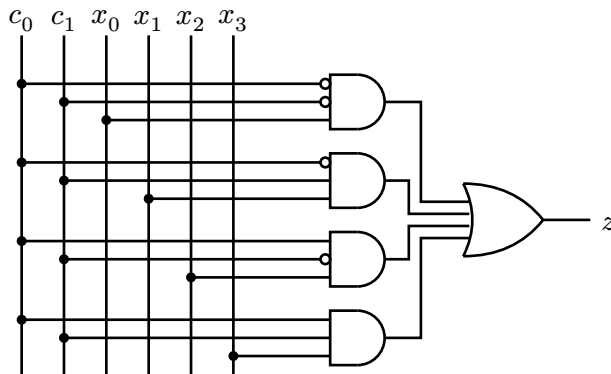
Una rete si può costruire in due modi: da zero con i cinque passi, oppure **componendo** reti già fatte. Esempio: un MUX che sceglie fra **quattro** ingressi con due bit di controllo $c_0 c_1$ (qui c_0 è quello più a sinistra). La barretta con un numero indica quanti bit passano su un filo.



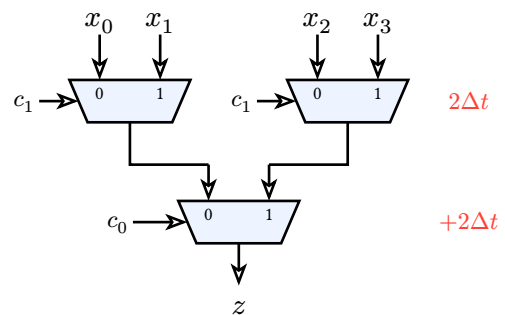
c_0	c_1	x_0	x_1	x_2	x_3	z
0	0	0	-	-	-	0
0	0	1	-	-	-	1
0	1	-	0	-	-	0
0	1	-	1	-	-	1
1	0	-	-	0	-	0
1	0	-	-	1	-	1
1	1	-	-	-	0	0
1	1	-	-	-	1	1

Da zero. Gli ingressi sono 6: la tabella avrebbe $2^6 = 64$ righe. Si accorcia scrivendo - dove il valore di un ingresso non conta: con controllo 00 conta solo x_0 . Ogni riga con tre - ne riassume $2^3 = 8$.

$$z = \overline{c_0} \overline{c_1} x_0 + \overline{c_0} c_1 x_1 + c_0 \overline{c_1} x_2 + c_0 c_1 x_3$$



Da zero: quattro AND e una OR, $2\Delta t$



Componendo tre MUX: $4\Delta t$

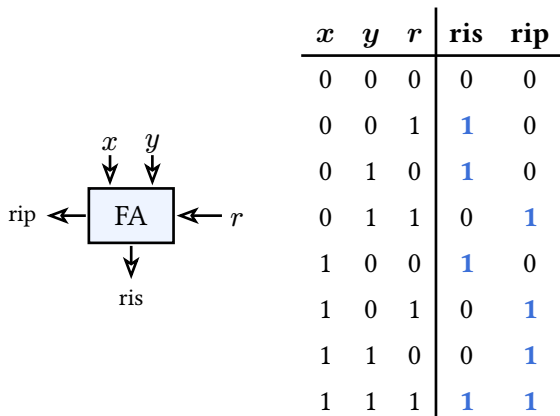
Componendo. Scegliere fra quattro vuol dire scegliere prima dentro ogni coppia, poi fra le due coppie. Bastano tre MUX a due ingressi: il bit meno significativo c_1 sceglie dentro le coppie, il più significativo c_0 sceglie la coppia. Non serve la tabella, ma il segnale attraversa due MUX di fila: $2\Delta t + 2\Delta t = 4\Delta t$. I due MUX in alto lavorano insieme, quindi contano una volta sola.

Regola generale: la rete progettata da zero ha un tempo di stabilizzazione minore o uguale a quella ottenuta componendo, perché è ottimizzata tutta insieme. Comporre però costa molta meno fatica.

2.3 Componenti

Alcune reti si usano così spesso che diventano mattoni pronti, come il MUX.

Il **full adder** (FA) fa una colonna della somma in binario: prende i due bit x e y e il riporto r che arriva dalla colonna a destra, e calcola il bit del risultato e il riporto per la colonna a sinistra.



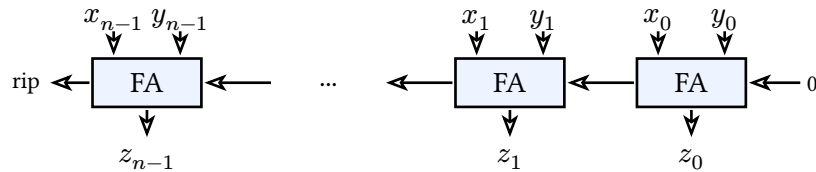
$$\begin{array}{r} 0100 \\ 0011 \\ + 0010 \\ \hline 0101 \end{array}$$

3 + 2: ogni colonna somma due bit e il riporto (in grigio)

$$\begin{aligned} \text{ris} &= \bar{x}\bar{y}r + \bar{x}y\bar{r} + x\bar{y}\bar{r} + xyr \\ \text{rip} &= \bar{x}yr + x\bar{y}r + xy\bar{r} + xyr \end{aligned}$$

Ogni formula ha quattro termini di tre variabili, tutto sotto 8: un livello di AND e uno di OR, quindi il FA si stabilizza in $2\Delta t$.

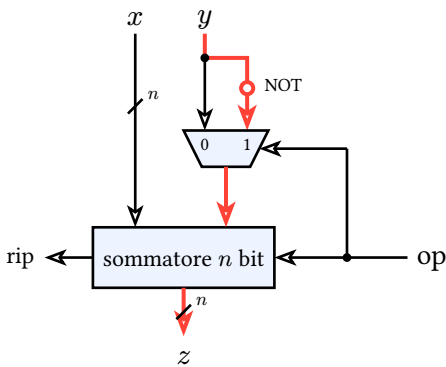
Per sommare numeri di n bit si mettono n FA **in cascata**: il riporto di ognuno entra nel successivo. Da qui in poi i bit si numerano col loro peso: x_0 è il meno significativo, x_{n-1} il più significativo.



Sommatore a n bit: ogni FA aspetta il riporto di quello alla sua destra

Ogni FA deve aspettare il riporto del precedente, quindi i ritardi **si sommano**: $n \cdot 2\Delta t = 2n\Delta t$. Con 2 bit sono $4\Delta t$. Se l'ultimo riporto vale 1, la somma di due numeri senza segno non sta in n bit.

Lo stesso sommatore fa anche la sottrazione: $x - y = x + \bar{y} + 1$. Un MUX sceglie fra y e $\bar{y}$, e lo stesso bit di controllo entra come riporto iniziale.



op = 0 somma, op = 1 sottrazione.
In rosso il cammino critico

Il tempo di stabilizzazione si legge sul **cammino critico** (*critical path*), il percorso più lento dagli ingressi alle uscite:

pezzo	ritardo
NOT	0
MUX a n bit	$2\Delta t$
sommatore a n bit	$2n\Delta t$
totale	$2(n+1)\Delta t$

Il MUX a n bit è fatto di n MUX a un bit con lo stesso controllo: lavorano tutti insieme, quindi costa $2\Delta t$ come uno solo.

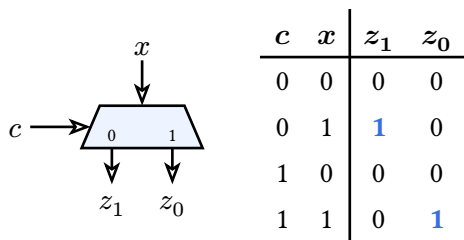
E progettando il sommatore da zero? La tabella non si scrive, ma il tempo si può stimare. Per due numeri di n bit:

- gli ingressi sono $2n$, quindi le righe sono 2^{2n} ;
- una colonna d'uscita ha al massimo $2^{2n} - 1$ uni (con tutti 1 sarebbe la costante 1), quindi l'OR ha circa 2^{2n} ingressi: servono $\lceil \log_8 2^{2n} \rceil = \lceil \frac{2n}{3} \rceil$ livelli, perché $\log_8 a = \frac{\log_2 a}{\log_2 8}$;
- ogni AND ha $2n$ ingressi: $\lceil \log_8 2n \rceil$ livelli.

Con 2 bit, contando anche il riporto iniziale, gli ingressi sono 5 e le righe 32: un OR di al massimo 31 termini (2 livelli) e AND da 5 ingressi (1 livello), cioè $3\Delta t$ contro i $4\Delta t$ dei due FA in cascata. Con $n = 8$: $\lceil \frac{16}{3} \rceil + \lceil \log_8 16 \rceil = 6 + 2 = 8\Delta t$, contro i $16\Delta t$ degli otto FA in cascata. Ma la tabella avrebbe $2^{16} = 65536$ righe: nessuno la scrive, si usano i FA.

Se in una colonna gli 1 sono più della metà, conviene scrivere i termini per gli **0** e negare l'uscita, tanto il NOT non costa: i termini sono al massimo la metà delle righe.

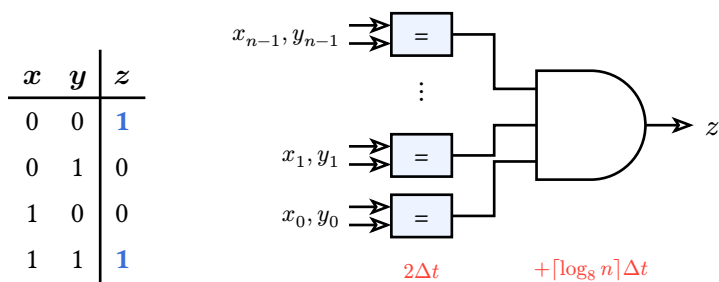
Il **demultiplexer** è il contrario del MUX: un ingresso x e più uscite. Il controllo sceglie su quale uscita mandare x ; le altre valgono 0. Servirà per scegliere in quale cella di memoria scrivere.



$$z_1 = \bar{c}x \quad z_0 = cx$$

Ogni uscita ha un solo 1, quindi un solo termine: l'OR non serve. C'è solo il livello di AND: $1\Delta t$.

Il **comparatore** dice se due numeri di n bit sono uguali: uscita 1 se sono uguali, 0 altrimenti. Per un bit la formula è $z = \bar{x}\bar{y} + xy$, che non si semplifica: $2\Delta t$. È il negato dello **XOR** (OR esclusivo), che vale 1 quando i due bit sono diversi. Per n bit si confronta ogni coppia x_i, y_i e si fa l'AND di tutte le risposte: basta una coppia diversa per dare 0. Avendo una porta già pronta per il confronto di un bit, con ritardo Δt , si guadagnerebbe solo $1\Delta t$: il livello dell'AND finale resta.



Comparatore a n bit: $(2 + \lceil \log_8 n \rceil)\Delta t$

Il **codificatore** ha n ingressi di cui **esattamente uno** vale 1, e $\log_2 n$ uscite che dicono in che posizione sta quell'1 (di solito 2^k ingressi e k uscite). Le altre combinazioni di ingressi non sono ammesse, quindi non compaiono nella tabella.

x_3	x_2	x_1	x_0	z_1	z_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

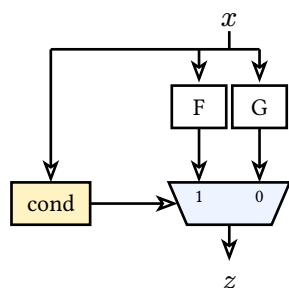
$$z_1 = \bar{x}_3 \bar{x}_2 \bar{x}_1 \bar{x}_0 + x_3 \bar{x}_2 \bar{x}_1 \bar{x}_0$$

$$z_0 = \bar{x}_3 \bar{x}_2 \bar{x}_1 \bar{x}_0 + x_3 \bar{x}_2 \bar{x}_1 \bar{x}_0$$

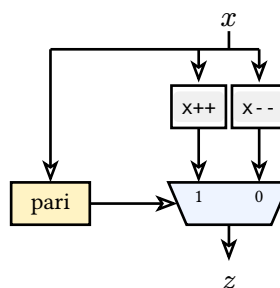
L'ultimo termine è uguale nelle due formule: si calcola una volta sola, 3 AND invece di 4. Il tempo resta $2\Delta t$, ma una porta in meno vuol dire meno spazio e meno consumo.

All'esame $2\Delta t$ è la risposta giusta solo se i livelli sono due. Conta sempre i livelli: il demultiplexer ne ha uno ($1\Delta t$), due FA in cascata ne hanno quattro ($4\Delta t$).

Costruire una rete componendo mattoni pronti, come nel circuito che somma e sottrae, si chiama **approccio strutturale**. Si parte da uno pseudocodice fatto di funzioni e scelte, e ogni pezzo diventa un componente: ogni funzione è una rete, la condizione è una rete con un bit d'uscita, e l'if è un MUX comandato da quel bit.



if (cond) f(x) else g(x)



if (pari(x)) x++ else x--

Nell'esempio di destra, con x di n bit, i tre blocchi si fanno con quello che c'è già:

- $x++$: un sommatore con ingressi x e la costante 1, riporto iniziale 0. Una costante è un filo fisso a 0 o a 1;
- $x--$: un sommatore con x e la costante 1 negata, riporto iniziale 1: è $x - 1$ fatto come sottrazione;
- $\text{pari}(x)$: un numero è pari quando il suo ultimo bit x_0 vale 0, quindi basta un NOT su x_0 , senza calcoli.

2.4 Semplificare le formule

Fra la somma di prodotti e la rete si può **semplificare** la formula con le regole dell'algebra. Si raccoglie, poi si usano $t + \bar{t} = 1$ e $x \cdot 1 = x$:

$$z = xyt + xy\bar{t} = xy(t + \bar{t}) = xy$$

Qui l'OR sparisce: da $2\Delta t$ a $1\Delta t$. Un termine si può anche usare due volte, perché $x + x = x$:

$$z = xyt + xy\bar{t} + \bar{x}yt = \underbrace{xyt + xy\bar{t}}_{xy} + \underbrace{xyt + \bar{x}yt}_{yt} = xy + yt$$

I livelli restano due, quindi il tempo non cambia, ma le porte calano: da 3 AND e 1 OR a 2 AND e 1 OR. Con porte da soli 2 ingressi il guadagno è più grande, perché ogni AND da tre ingressi diventa due porte: da 8 porte a 3. Raccogliendo ancora, $z = y(x + t)$: 2 porte.

Semplificare quindi non sempre fa guadagnare tempo, ma fa risparmiare porte: meno spazio e meno consumo. Allo stesso modo il riporto del FA, usando tre volte il termine xyr , diventa:

$$\text{rip} = yr + xr + xy$$

Per vedere cosa raccogliere si usano le **mappe di Karnaugh**: sono la tabella di verità ridisegnata come griglia, con alcune variabili sulle colonne e le altre sulle righe. Le combinazioni sono scritte nell'ordine 00, 01, 11, 10, così fra due celle vicine cambia **un solo bit**.

		xy	
		00	01
x	y	0	1
	0	1	0
		1	1

		xy				
		00	01	11	10	
x	y	00	01	11	10	
	00	1	1	0	0	
	01	1	1	0	1	
	11	0	0	0	0	
		10	1	1	1	1

		xy				
		00	01	11	10	
x	y	00	01	11	10	
	00	1	1	0	0	
	01	1	1	0	1	
	11	0	0	0	0	
		10	1	1	1	1

2 variabili:

il comparatore a un bit

4 variabili

gli stessi 1 raccolti in tre gruppi

Sulla mappa si cercano **gruppi di 2^k celle vicine, tutte a 1, a forma di quadrato o rettangolo**. Ogni gruppo diventa un solo termine: ci restano le variabili che nel gruppo non cambiano, le altre k spariscono.

- Gruppo **blu**, 4 celle: y e b cambiano, x e a valgono sempre 0. Termine $\bar{x}\bar{a}$.
- Gruppo **verde**, 4 celle: tutta la riga $ab = 10$, x e y cambiano. Termine $a\bar{b}$.
- Gruppo **rosso**, 2 celle: la mappa **si richiude sui bordi**, la prima colonna è vicina all'ultima (e la prima riga all'ultima). Cambia solo x . Termine $\bar{y}\bar{a}b$.

$$z = \bar{x}\bar{a} + a\bar{b} + \bar{y}\bar{a}b$$

Un 1 isolato resta un termine con tutte le variabili; su 4 variabili un gruppo da 8 ne lascia una sola. Coprendo tutti gli 1 con il minor numero di gruppi, i più grandi possibili, si ottiene la formula più semplice. I gruppi si chiamano **implicanti**.

Le due uscite del full adder:

xy		00	01	11	10
r	0	0	1	0	1
	1	1	0	1	0

ris: gli 1 sono tutti isolati,
restano i quattro termini da tre variabili

xy		00	01	11	10
r	0	0	0	1	0
	1	0	1	1	1

rip: tre gruppi da 2,
rip = $xy + yr + xr$

Nel codificatore molte combinazioni di ingressi non sono ammesse: sulla mappa si segnano con -. Non capiteranno mai, quindi si possono contare come 1 quando fa comodo per ingrandire un gruppo.

x_1x_0		00	01	11	10
x_3x_2	00	-	0	-	0
	01	1	-	-	-
	11	-	-	-	-
	10	1	-	-	-

$$z_1 = x_3 + x_2$$

x_1x_0		00	01	11	10
x_3x_2	00	-	0	-	1
	01	0	-	-	-
	11	-	-	-	-
	10	1	-	-	-

$$z_0 = x_3 + x_1$$

Due gruppi da 8 per ogni uscita: spariscono tre variabili su quattro, e ogni uscita diventa un solo OR.

Con 5 variabili servirebbero due mappe 4×4 una sopra l'altra, una per ogni valore della quinta variabile. Per questo oltre le 4 variabili le mappe non si usano.